

Thai Poetry Rhyme Verification using Hybrid Rules Based Approach for Education*

*Note: Sub-titles are not captured for <https://ieeexplore.ieee.org> and should not be used

1st Warit Yuvaniyama

School of Information, Computer, and Communication Technology
Sirindhorn International Institute of Technology, Thammasat University
Pathum Thani 12120, Thailand
<https://orcid.org/0009-0007-1630-4286>

2nd Athit Phinyachok

School of Information, Computer, and Communication Technology
Sirindhorn International Institute of Technology, Thammasat University
Pathum Thani 12120, Thailand
<https://orcid.org/0009-0000-3783-8104>

3rd Gunn Sanguankittipun

School of Information, Computer, and Communication Technology
Sirindhorn International Institute of Technology, Thammasat University
Pathum Thani 12120, Thailand
<https://orcid.org/0009-0005-8812-4992>

4th Leo Colombo

School of Information, Computer, and Communication Technology
Sirindhorn International Institute of Technology, Thammasat University
Pathum Thani 12120, Thailand
<https://orcid.org/0009-0004-9133-5398>

5th Prachya Boonkwan*

School of Information, Computer, and Communication Technology
Sirindhorn International Institute of Technology, Thammasat University
Pathum Thani 12120, Thailand
<https://orcid.org/0000-0003-1405-1071>*

Abstract—

Index Terms—Thai NLP, Poetry Generation, Rule-based Verification, Natural Language Processing, Reinforcement Learning, Educational Technology

I. Introduction

- Context: Introduce Thai Klon-Paed (โคลงแปด) and its strict structural rules (Sumpus, Mattra, Sara).
- The Core Problem: Generative AI and LLMs are probabilistic, making them inherently bad at strict deterministic constraints like syllables and rhymes.
- The Dual Need:
 - Research: To train an LLM to write poetry (e.g., via RLHF/RLAIF), we need a flawless, programmatic reward environment to score rhymes.
 - Education: Teachers need zero-hallucination, interactive grading tools for students.

- Limitations of Prior Art: Previous checkers (PyThaiNLP 5.0.1) and G2P models (Kongfha) fail on edge cases, drop stanzas, or are computationally too slow for RL loops.
- Contributions:
 - Improved core verification algorithms (`check_sara`, `check_mattra`, `is_sumpus`).
 - Creation of a 36,475-stanza gold-standard evaluation dataset.
 - Integration into PyThaiNLP 5.3.5 and an open-source Hugging Face educational application.

II. Background and Related Work

A. Linguistic Rules of Klon-Paed

- Briefly define the requirements for a valid rhyme in Thai: matching vowel sound (`check_sara`) and matching final consonant class (`check_mattra`).

- Define inter-stanza (rX) and intra-stanza (r1, r2, r3) connections.

B. Existing Verification Systems

- Model A (Old PyThaiNLP 5.0.1): Dictionary-based, lacks the r3 rule, fails on complex morphology.
- Model C (Kongfha / tlk G2P): Romanization-based. Extremely slow (single-threaded) and collapses short/long vowels, leading to false positives.

III. Methodology: Hybrid Verification Architecture

A. Core Rule-Based Improvements (Model B)

- Explain the mechanics of Model B (now PyThaiNLP 5.3.5).
- Phonetic Normalization: Handling transformed/reduced vowels (,).
- Edge Case Handling:
 - handle_karun_sound_silence: Stripping silenced characters (e.g., " " → " ").
 - _is_true_final: Differentiating genuine finals (- , -) from clusters or digraphs.

B. Dictionary-Enhanced Verification (Model D)

- Explain the Klonpad variant. While Model B acts as a general-purpose oracle, it has "blind spots" (e.g., the silent ' ' in " ").
- Introduce the G2P Override Dictionary (built on Phra Aphai Mani) to fix these specific phonetic misreadings.
- Ablation Study of Fallbacks: Briefly discuss why D_ssg (SSG fallback) outperformed D_w2p (W2P fallback). W2P introduces hallucinations on short words, hurting overall accuracy.

IV. Dataset Construction and Augmentation

A. Gold-Standard Corpus Compilation

- Data sourced from vajirayana.org (Khun Chang Khun Phaen, Phra Aphai Mani, etc.).
- Cleaning pipeline resulting in 191 files, 36,475 stanzas, and 145,709 total rhyme checks.
- Highlight the data-completeness fix (retaining ≥ 10 syllable waks) that restored inter-stanza (rX) chains.

B. Targeted Data Augmentation

- Explain the need for precision and recall probes.
- Negative Sampling (Precision Probes): 10,000 instances. Discuss operators like C3_short_long (vowel length swaps) and C9_old_accept (traps for Model A).
- Positive Sampling (Recall Probes): 1,200 instances. Discuss "Oracle-blind" positives (e.g., ,) intentionally designed to deduct points from Model B where it fails linguistically.

V. Evaluation and Results

A. Experimental Setup

- Define the evaluated models (A, B, C, D_w2p, D_ssg) and metrics (Precision, Recall, F1, Coverage).
- Mention computational mitigations for Model C (persistent worker pool, G2P caching).

B. Gold Recall Analysis

- Present the Stanza-level gold recall table.
- Highlight that D_ssg achieves the highest recall (88.5%) followed by B (87.5%), vastly outperforming A (73.4%).

C. Augmentation and Error Analysis

- Present the Augmentation-only overall table.
- Precision vs. Recall Trade-off: Explain that Model B acts as a perfect oracle (0 FPs on negatives) but misses oracle-blind positives. Model C catches rhymes but has 3,129 FPs (mostly on C3_short_long).
- The Override Advantage: Show how Model D_w2p recovers 95.7% of the oracle-blind spots (e.g., reading →) that Model B misses.

VI. Applications and Future Work

A. Educational Impact

- Discuss the interactive Hugging Face web interface.
- Explain how deterministic, explainable feedback aids teachers and students in learning Klon 4/8 structures without LLM hallucinations.

B. Foundation for Generative AI

- Discuss how this verifier is currently implemented as the Reward Model environment for Reinforcement Learning (RL) to train a small LLM to write Klon-Paed.

VII. Conclusion

- Summarize: The hybrid approach effectively marries the reliability of rule-based logic with the edge-case coverage of G2P overrides.
- Final impact statement: The successful integration of Model B into PyThaiNLP 5.3.5 sets a new standard for Thai computational linguistics, paving the way for both educational tools and advanced LLM pipelines.

Acknowledgment

- Acknowledge vajirayana.org for the literary datasets.
- Acknowledge the PyThaiNLP maintainers for review-ing and merging the PR.
- Acknowledge the developers of the tlk and KlonSuphap-LM baselines.

References

- [1] PyThaiNLP Contributors, "PyThaiNLP: Thai Natural Language Processing in Python," [Online]. Available: <https://github.com/PyThaiNLP/pythainlp>
- [2] Kongfha, "KlonSuphap-LM," [Online]. Available: [Insert Link]
- [3] [Add other relevant Thai NLP, G2P, or LLM RLHF papers here]